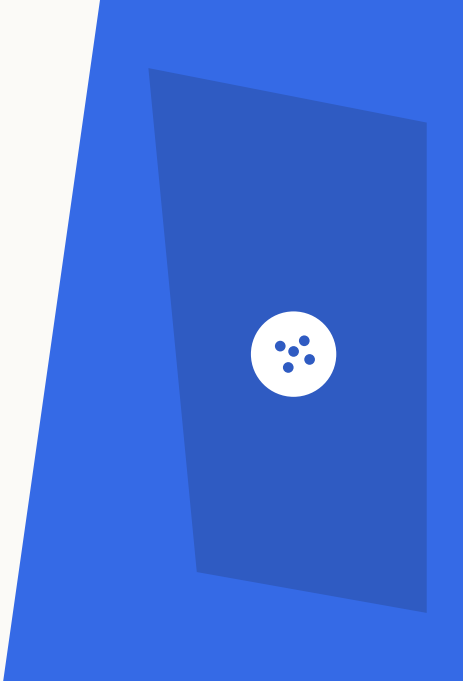
BASH SCRIPTING FUNDAMENTALS

# Errors & Debugging

## Exit codes, strict mode, traps, and tracing

Merit Advisory
Bash Scripting Fundamentals Bootcamp
Beginner-friendly lecture materials
July 2026



WHAT WE WILL COVER

# Agenda

1. **Section 1: Exit codes and status basics** — 4 slides
2. **Section 2: Status in control flow** — 4 slides
3. **Section 3: Strict mode** — 4 slides
4. **Section 4: Intentional failures** — 4 slides
5. **Section 5: Arguments and preflight checks** — 4 slides
6. **Section 6: Traps and cleanup** — 4 slides
7. **Section 7: Logging and runtime switches** — 4 slides
8. **Section 8: Tracing with xtrace** — 4 slides

9 **Section 9: Syntax checks and ShellCheck** — 4 slides

10 **Section 10: Small tests and checkpoints** — 4 slides

11 **Section 11: Resilience patterns** — 4 slides

12 **Section 12: Production mistakes and final checks** — 4 slides

# 1 Section 1

Exit codes and status basics

# 1.1 Exit codes are program results · Exit 0 means success

- **Every** command finishes with an integer status
- **Zero** means success by long Unix convention
- **Nonzero** means the command reported a problem
- Use 0 when the script completed its contract
- A successful script should be quiet or useful
- Callers can branch on the zero result

## Examples

```
# Successful command
pwd
echo $?
# Failing command
ls /missing
echo $?
echo done
exit 0
if ./build.sh; then echo ok; fi
```

# 1.2 Exit 1 means general failure · Exit 2 means bad usage

- Use 1 for ordinary runtime failures
- Keep the error message specific
- Reserve special codes for clearer categories
- Use 2 when arguments or options are invalid
- Print usage before exiting
- Bad usage is different from runtime failure

## Examples

```
# Report failure
echo 'missing input' >&2
exit 1
# Caller sees failure
./job.sh || echo failed
# Usage error
echo 'usage: tool FILE' >&2
exit 2
[ "$#" -eq 1 ] || exit 2
```

# 1.3 Exit 126 means not executable · Exit 127 means command not found

- **The** command was found but could not run
- **Permissions** are the common cause
- Fix with chmod only when execution is intended
- The shell could not locate the command name
- Check spelling and PATH
- Dependency checks prevent surprises later

### Examples

```
# Permission failure
touch script.sh
./script.sh
# Make executable
chmod +x script.sh
./script.sh
not_a_command
echo $?
command -v tar || echo missing
```

# 1.4 $? must be read immediately

- The status variable changes after every command
- Even echo overwrites the previous status
- Save it right away if needed later

### Examples

```
# Capture status
grep ERROR app.log
status=$?
# Overwritten status
grep ERROR app.log
echo checked
# ...
```

# 2 Section 2

Status in control flow

# 2.1 true always succeeds · false always fails

- **true** is useful for tests and placeholders
- It returns status 0 every time
- Loops can use **true** for intentional infinity
- **false** returns status 1 every time
- **It** is useful for exercising failure paths
- It makes examples deterministic

### Examples

```
# Status 0
true
echo $?
# Intentional loop
while true; do date; break; done
# Status 1
false
echo $?
if false; then echo yes; else echo no; fi
```

# 2.2 if reads command status · && runs after success

- Bash tests success and failure directly
- No separate boolean type is required
- The then branch runs only on status 0
- The **right** command runs only if the left succeeds
- Use it for simple dependent steps
- Keep long chains readable

### Examples

```
# Directory exists
if cd /tmp; then pwd; fi
# Command as condition
if grep -q root /etc/passwd; then echo found; fi
# Dependent command
mkdir -p out && echo ready
# Stop chain
false && echo never
```

# 2.3 || runs after failure · Status propagation matters

- The **right** command runs only if the left fails
- Use it for fallback or error handling
- **It** is common with small die helpers
- Functions and scripts return the last command **status**
- An accidental final echo can hide failure
- Return explicitly when the **status** is the contract

### Examples

```
# Fallback
cd project || exit 1
# Message on failure
cp a b || echo copy failed
# Hidden failure
check() { false; echo done; }
check; echo $?
check() { false; return $?; }
check
```

# 2.4 Use command || die

- A **die** helper keeps fatal failures consistent
- The **failing** command stays next to its message
- Exit nonzero after writing to stderr

### Examples

```
# Define die
die() { echo "$*" >&2; exit 1; }
# Use die
cp "$src" "$dst" || die 'copy failed'
```

# 3 Section 3

Strict mode

# 3.1 set -e exits on many failures · errexit is ignored in if tests

- **errexit** stops the script after many failed commands
- It reduces accidental continuation
- It still has important exceptions
- A failing condition is expected control flow
- **set** -e does not exit inside the if test
- Handle the else branch intentionally

### Examples

```
# Enable errexit
set -e
cp missing out
# Local enable
set -e
build_project
set -e
if grep -q x file; then echo found; fi
if cd app; then pwd; else echo no; fi
```

# 3.2 errexit has && and || exceptions · set -u catches unset variables

- Failures before && or || are part of branching
- Bash does not exit for those expected statuses
- Avoid hiding unexpected failures in complex lists
- nounset fails when an unset variable is expanded
- It catches typos early
- Use defaults when missing values are acceptable

**Examples**

```
# Allowed failure
set -e
false || echo handled
# Success chain
set -e
mkdir -p out && touch out/a
set -u
echo "$missing"
echo "${name:-guest}"
```

# 3.3 nounset pitfalls are predictable · pipefail exposes pipeline failures

- Optional variables need default syntax
- Empty and unset are not the same idea
- Arrays and positional parameters need care
- Without **pipefail** only the last command decides
- Earlier pipeline failures can be masked
- **pipefail** returns failure when any stage fails

## Examples

```
# Require value
echo "${API_TOKEN:?missing token}"
# Optional first arg
first="${1:-}"
echo "$first"
# Masked failure
grep x missing | sort
echo $?
set -o pipefail
grep x missing | sort
```

# 3.4 Use set -euo pipefail together

- The trio catches failed commands and missing values
- It is a strong default for new scripts
- Add exceptions only where you mean them

**Examples**

```
# Strict header
set -euo pipefail
# Handled exception
grep -q x file || true
```

# 4 Section 4

Intentional failures

# 4.1 Use || true only intentionally · grep no match is status 1

- Some commands fail for acceptable reasons
- Document why the failure is harmless
- Avoid using it as a blanket silencer
- **grep** uses 1 for no matching lines
- No match is often data, not a crash
- Handle it separately from file errors

### Examples

```
# Optional cleanup
rm -f old.tmp || true
# Expected no match
grep -q TODO file || true
# No match allowed
if grep -q ERROR log; then echo bad; fi
# Count with fallback
grep -c ERROR log || true
```

# 4.2 Fail early for required steps · Accumulate nonfatal errors

- Stop when later steps depend on earlier success
- Early failure keeps damage small
- Strict mode supports this habit
- Some scripts should process everything possible
- Track failures and report them at the end
- Return nonzero if any item failed

**Examples**

```
# Required copy
cp config.yml out/config.yml
# Required directory
cd app || exit 1
# Count failures
errors=0
cp a b || errors=$((errors+1))
[ "$errors" -eq 0 ] || exit 1
```

# 4.3 return exits a function · exit ends the shell script

- **return** sets the function status
- It does not **exit** the whole script
- Use it for reusable checks
- **exit** terminates the current shell process
- Use it for fatal top-level errors
- Avoid **exit** inside libraries meant to be sourced

### Examples

```
# Return failure
check() { [ -f "$1" ] || return 1; }
# Call check
check config.yml || echo missing
# Fatal error
[ -f config.yml ] || exit 1
# Explicit code
exit 2
```

# 4.4 Strict mode in sourced files needs care

- A sourced file changes the caller shell
- set options can leak to the importing script
- Prefer functions that return instead of exiting

**Examples**

```
# Sourcing changes shell
. ./lib.sh
# Library function
load_config() { [ -f "$1" ] || return 1; }
```

# 5 Section 5

Arguments and preflight checks

# 5.1 Check the number of arguments · Put usage in a function

- $# contains the positional argument count
- Validate before using $1 or $2
- Bad argument count usually exits 2
- A usage function keeps help text consistent
- Write usage to stderr for errors
- Call it before exit 2

### Examples

```
# Require one arg
[ "$#" -eq 1 ] || exit 2
# Require two args
[ "$#" -eq 2 ] || { echo usage >&2; exit 2; }
# Usage function
usage() { echo 'usage: app FILE' >&2; }
# Use it
[ "$#" -eq 1 ] || { usage; exit 2; }
```

# 5.2 Validate option values · Require important environment variables

- Options can be present but invalid
- Check allowed values near parsing
- Fail before doing work
- Environment variables are inputs too
- Use parameter expansion for clear errors
- Do not wait for a later command to fail vaguely

## Examples

```
# Mode check
[ "$mode" = fast ] || [ "$mode" = safe ]
# Reject value
echo 'invalid mode' >&2
exit 2
# Require env
: "${API_TOKEN:?API_TOKEN required}"
LOG_LEVEL="${LOG_LEVEL:-info}"
```

# 5.3 need checks dependencies · Assert a file exists

- Check external commands before the main work
- command -v is portable and quiet
- Dependency errors should name the missing tool
- Use file tests for required inputs
- Quote the path being tested
- Report the exact missing path

## Examples

```
# Define need
need() { command -v "$1" >/dev/null || exit 127; }
# Check tools
need tar
need gzip
# File assertion
[ -f "$file" ] || { echo "missing: $file" >&2; exit 1; }
[ -d "$dir" ] || exit 1
```

# 5.4 Assert a directory is writable

- Many scripts fail later when output is not writable
- Test the output directory before generating files
- Create missing directories explicitly

**Examples**

```
# Writable check
[ -w "$outdir" ] || exit 1
# Create first
mkdir -p "$outdir"
[ -w "$outdir" ]
```

# 6 Section 6

Traps and cleanup

# 6.1 trap EXIT always runs at script end · Clean up temp directories

- EXIT traps run on success and failure
- Use them for cleanup and final messages
- Define the **trap** after resources exist
- mktemp -d creates a unique workspace
- An EXIT **trap** removes it even on failure
- Quote the temp path in cleanup

## Examples

```
# Exit trap
cleanup() { rm -f "$tmp"; }
trap cleanup EXIT
# Final message
trap 'echo done >&2' EXIT
# Temp dir
tmp=$(mktemp -d)
trap 'rm -rf "$tmp"' EXIT
cp input "$tmp/input"
```

# 6.2 trap INT handles Ctrl-C · trap ERR handles many failures

- **INT** is sent by an interactive interrupt
- Use it to print a clear cancellation message
- Exit with nonzero status after interruption
- ERR traps run when errexit would react
- **They** are useful for logging failure context
- They follow the same caveats as set -e

## Examples

```
# Interrupt trap
trap 'echo cancelled >&2; exit 130' INT
# Long task
sleep 30
# ERR trap
trap 'echo failed at $LINENO >&2' ERR
set -e
false
```

# 6.3 ERR trap caveats mirror errexit · errtrace carries ERR into functions

- **ERR** is skipped in expected failure positions
- if tests and || handlers do not trigger it
- Do not treat **ERR** as full exception handling
- set -E enables **ERR** inheritance in functions
- **It** is also written as set -o errtrace
- Use it with strict scripts that log failures

### Examples

```
# No ERR in if
set -Eeo pipefail
if false; then echo ok; fi
# No ERR when handled
set -Eeo pipefail
false || echo handled
set -Ee
trap 'echo ERR $LINENO >&2' ERR
work() { false; }
work
```

# 6.4 inherit_errexit helps command substitutions

- Command substitutions can weaken errexit behavior
- **inherit_errexit** keeps failure behavior stricter
- **It** is a Bash option, not POSIX sh

### Examples

```
# Enable option
shopt -s inherit_errexit
# Substitution
value=$(failing_command)
```

# 7 Section 7

Logging and runtime switches

# 7.1 Log errors to stderr · Add timestamps to logs

- **stdout** is for normal script output
- **stderr** is for diagnostics and errors
- This keeps pipelines clean
- Timestamps make production reports easier to follow
- Use a consistent format
- Keep the logging function small

### Examples

```
# Error message
echo 'copy failed' >&2
# Normal output
echo "$result"
# Timestamp
date '+%Y-%m-%d %H:%M:%S'
# Log line
echo "$(date '+%F %T') start" >&2
```

# 7.2 Use log levels · Verbose flags reveal progress

- Levels separate info from warnings and errors
- Simple prefixes help when reading files
- Do not overbuild logging for small scripts
- **Verbose** mode is useful for humans running scripts
- Keep default output concise
- Guard **verbose** messages with a variable

### Examples

```
# Info
echo 'INFO starting' >&2
# Error
echo 'ERROR missing file' >&2
# Verbose check
[ "${verbose:-0}" -eq 1 ] && echo working >&2
verbose=1
[ "$verbose" -eq 1 ] && echo step >&2
```

# 7.3 Debug environment variables are quick toggles · Dry-run flags prevent changes

- Environment variables can enable debugging without parsing
- Default them safely under set -u
- Document supported **debug** variables
- Dry runs show what would happen
- **They** are safest when routed through one helper
- **They** are excellent for destructive scripts

## Examples

```
# Debug toggle
[ "${DEBUG:-0}" = 1 ] && set -x
# Run with debug
DEBUG=1 ./backup.sh
# Dry-run variable
dry_run=1
# Guard action
[ "$dry_run" = 1 ] && echo rm file || rm file
```

# 7.4 Echo commands before running risky work

- A run helper makes command logging consistent
- It pairs naturally with dry-run mode
- Use arrays for complex real scripts

### Examples

```
# Run helper
run() { echo "+ $*" >&2; "$@"; }
# Use helper
run mkdir -p out
```

# 8 Section 8

Tracing with xtrace

# 8.1 bash -x traces a script · set -x starts tracing inside a script

- xtrace prints commands after expansion
- It helps reveal what **Bash** is really running
- Use it on small reproductions when possible
- Enable tracing around the suspicious area
- Tracing everything can be noisy
- Turn it off after the section

### Examples

```
# Trace script
bash -x ./script.sh
# Trace with args
bash -x ./script.sh input.txt
# Start trace
set -x
cp "$src" "$dst"
set -x
work
```

# 8.2 set +x stops tracing · PS4 customizes trace prefixes

- Stop tracing before secrets or noisy loops
- Keep traces focused and readable
- Pair every temporary **set** -x with set +x
- **PS4** controls the prefix before traced commands
- Include line numbers for faster debugging
- Use single quotes so variables expand during tracing

## Examples

```
# Stop trace
set +x
# Around block
set -x
build
# ...
PS4='+ $LINENO: '
set -x
PS4='+ ${FUNCNAME[0]}:$LINENO: '
```

# 8.3 Debug functions one at a time · Protect secrets while tracing

- Small functions are easier to trace
- Trace the function call and its inputs
- Confirm the function status after it returns
- xtrace prints expanded values
- Disable tracing around tokens and passwords
- Prefer secret-safe diagnostic messages

**Examples**

```
# Trace call
set -x
make_report "$file"
# ...
# Check status
make_report "$file"
echo $?
set +x
curl -H "Auth: $TOKEN" url
set -x
next_step
```

# 8.4 Make a minimal reproduction

- Copy only the failing lines into a tiny script
- Use fixed sample input
- Debug faster before returning to the full script

### Examples

```
# Tiny script
printf '%s
' a b | grep c
# Trace it
bash -x repro.sh
```

# 9 Section 9

Syntax checks and ShellCheck

# 9.1 bash -n checks syntax · ShellCheck finds common bugs

- No commands are executed
- It catches parse errors before runtime
- Run it before testing behavior
- **ShellCheck** is a static analyzer for shell scripts
- It catches quoting and portability issues
- Treat warnings as teaching moments

**Examples**

```
# Syntax check
bash -n script.sh
# Check all scripts
bash -n scripts/*.sh
# Run ShellCheck
shellcheck script.sh
# Several files
shellcheck bin/*.sh
```

# 9.2 SC2086 warns about unquoted expansions · SC2154 warns about unknown variables

- Unquoted variables can split into many words
- They can also expand globs accidentally
- Quote variables unless you intentionally want splitting
- Typos in variable names are common
- ShellCheck catches many before runtime
- set -u catches the rest during testing

### Examples

```
# Risky
cp $src $dst
# Safer
cp "$src" "$dst"
# Typo
echo "$ouput"
# Correct
echo "$output"
```

# 9.3 SC2164 warns about unchecked cd · SC2046 warns about command splitting

- A failed cd can make later commands run elsewhere
- Check cd or let strict mode stop the script
- Use subshells for temporary directory changes
- **Unquoted** command substitution can split output
- Prefer arrays or while read loops
- Quote substitution when one string is intended

## Examples

```
# Unchecked
cd "$dir"
rm -f *.tmp
# Checked
cd "$dir" || exit 1
# Risky
rm $(cat files.txt)
name="$(cat name.txt)"
```

# 9.4 Ignore warnings only with a reason

- Some warnings are false positives for your intent
- Keep ignores narrow
- Leave a short explanation for future readers

### Examples

```
# Inline ignore
# shellcheck disable=SC2086
cmd $flags
# File check
shellcheck script.sh
```

# 10 Section 10

Small tests and checkpoints

# 10.1 Compare expected output · Write small test scripts

- A tiny expected file makes behavior concrete
- diff returns nonzero when output differs
- **This** is enough for many script tests
- Tests can be plain Bash at first
- Each test should set up its own inputs
- Exit nonzero when an assertion fails

**Examples**

```
# Capture output
./report.sh > actual.txt
# Compare
diff -u expected.txt actual.txt
# Test file
bash tests/report_test.sh
# Fail assertion
[ -s report.txt ] || exit 1
```

# 10.2 Use temp fixtures · Assert exit codes

- Fixtures keep tests repeatable
- Temporary directories avoid polluting the project
- Clean them with traps
- Some tests care more about status than output
- Capture the status immediately
- Test both success and failure paths

## Examples

```
# Fixture dir
tmp=$(mktemp -d)
trap 'rm -rf "$tmp"' EXIT
# Fixture file
printf 'ERROR x
' > "$tmp/app.log"
./tool missing
status=$?
[ "$status" -eq 2 ] || exit 1
```

# 10.3 Test error messages · Add checkpoint commands

- Helpful errors are part of script behavior
- Capture stderr separately
- Keep messages stable enough to **test**
- Checkpoint commands prove assumptions while debugging
- Use pwd, ls, and printf deliberately
- Remove or guard checkpoints before shipping

**Examples**

```
# Capture stderr
./tool missing 2>err.txt
# Check message
grep -q usage err.txt
# Location check
pwd >&2
printf 'file=%s
' "$file" >&2
```

# 10.4 Run a smoke test in CI

- A smoke test catches completely broken scripts
- **Run** syntax checks before behavior tests
- Keep the first CI version simple

### Examples

```
# Syntax smoke
bash -n script.sh
# Behavior smoke
./script.sh --help >/dev/null
```

# 11 Section 11

Resilience patterns

# 11.1 Retry transient commands · Backoff reduces pressure

- Network and service commands can fail temporarily
- **Retry** only operations that are safe to repeat
- Limit attempts so failures still surface
- Waiting longer between attempts avoids hammering services
- Simple linear **backoff** is often enough
- Log each **retry** so delays are visible

**Examples**

```
# Retry loop
for i in 1 2 3; do curl -fsS url && break; sleep 1; done
# Attempt count
attempts=3
# Backoff sleep
sleep "$i"
# Retry message
echo "retry $i" >&2
```

# 11.2 timeout prevents hangs · Capture network status clearly

- External commands can hang forever
- **timeout** stops a command after a limit
- Treat **timeout** as a normal failure to handle
- curl can fail for HTTP or connection reasons
- Use flags that make failures visible
- Report the URL or operation that failed

### Examples

```
# Limit runtime
timeout 10s curl -fsS url
# Handle timeout
timeout 5s ./job.sh || echo timed out
# Curl fail fast
curl -fsS "$url" -o out.json
# Handle curl
curl -fsS "$url" || die 'download failed'
```

# 11.3 pipefail motivation is data safety · PIPESTATUS shows each stage

- A report can look valid after an earlier pipeline error
- **pipefail** prevents partial data from seeming successful
- Use it for reporting and deployment scripts
- **PIPESTATUS** stores statuses for the last pipeline
- It must be read immediately
- **It** is useful for debugging pipeline failures

### Examples

```
# Unsafe report
grep ERROR missing.log | sort > report.txt
# Safer report
set -o pipefail
grep ERROR missing.log | sort > report.txt
# Inspect pipeline
grep x file | sort
echo "${PIPESTATUS[*]}"
statuses=("${PIPESTATUS[@]}")
```

# 11.4 Cleanup on failure protects reruns

- Partial files can confuse the next run
- Write to temp files before moving into place
- Traps and atomic moves pair well

## Examples

```
# Temp output
tmp=$(mktemp)
trap 'rm -f "$tmp"' EXIT
# Atomic finish
mv "$tmp" report.txt
```

12

# Section 12

Production mistakes and final checks

# 12.1 Missing quotes cause surprise words · Pipelines can mask failures

- Spaces in paths become multiple arguments
- Globs can expand unexpectedly
- Quoting prevents many production bugs
- The **final** command may succeed while earlier work failed
- Reports and filters are common victims
- Use pipefail in serious scripts

## Examples

```
# Broken path
rm $backup_dir/file
# Quoted path
rm "$backup_dir/file"
# Masked
cat missing | wc -l
set -o pipefail
cat missing | wc -l
```

# 12.2 Do not lose the real status · Unchecked cd is dangerous

- Logging after a command changes $?
- Save status before printing diagnostics
- Return or exit with the saved value
- A failed cd leaves the script in the old directory
- Destructive commands can hit the wrong place
- Check cd or run inside a subshell

### Examples

```
# Save first
./job.sh
status=$?
# Exit saved
echo "status=$status" >&2
exit "$status"
# Check cd
cd "$dir" || exit 1
( cd "$dir" && rm -f *.tmp )
```

# 12.3 set -u and arrays need defaults · Race-free temp files matter

- Empty arrays and unset arrays behave differently
- Use safe expansion when an array may be empty
- Test scripts with no inputs
- Predictable temp names can collide
- mktemp creates safer unique paths
- Clean temporary resources automatically

## Examples

```
# Safe args
args=("${extra_args[@]:-}")
# Optional scalar
name="${name:-}"
# Bad temp
tmp=/tmp/report.txt
tmp=$(mktemp)
trap 'rm -f "$tmp"' EXIT
```

# 12.4 Use a final debugging checklist

- Run bash -n before behavior tests
- Run ShellCheck before sharing
- Test success, bad usage, and one failure case

### Examples

```
# Checklist commands
bash -n script.sh
shellcheck script.sh
# Failure test
./script.sh missing || echo failed
```

NEXT STEP

# Practice every example in your terminal

## Then open the next module deck

Merit Advisory
07 · Errors